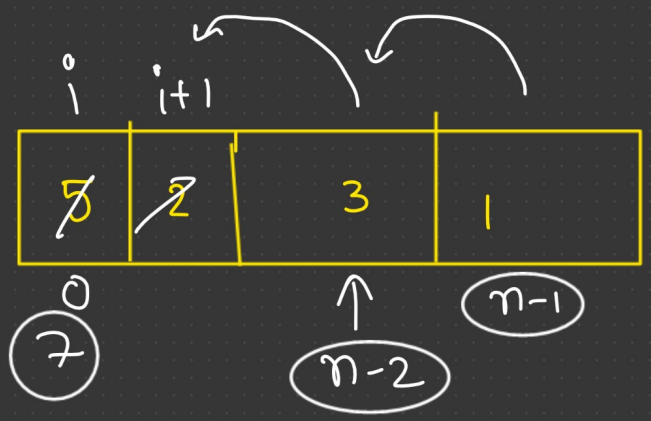
operation = ~~0~~ ~~1~~ ~~2~~ 3

1) 6 4
6, 2, 4, 3, 1
8 7

2) 6
6, 2, 4, 4
8 8

3) 6, 6, 4

4) 6, 10 ✓



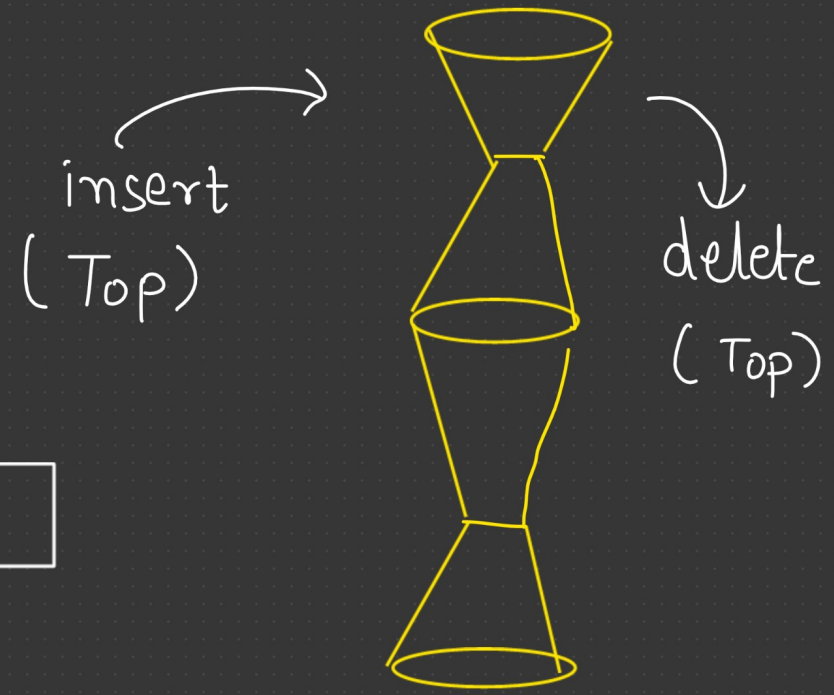
array size = n

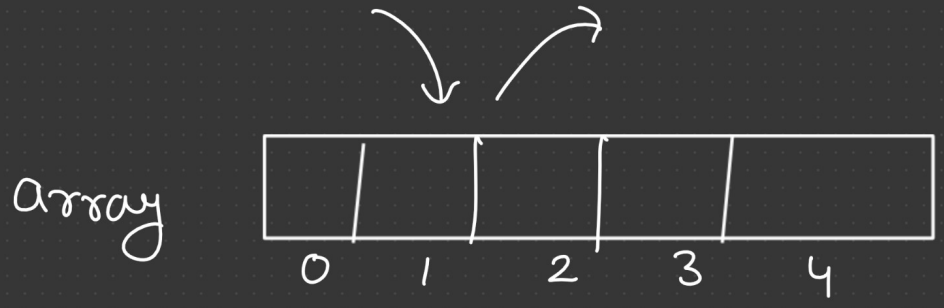
```
for ( int i= 0 ; i< n-1 ; i++)
{
    if ( nums[i] > nums[i+1])
      nums[i]= n[i] + num[i+1];

    for( int j= i+1 ; j<n-1 ; j++)
      {
          num[j] = num[j+1];
      }
    n--;
    Operation++;

    break; ?
}
```

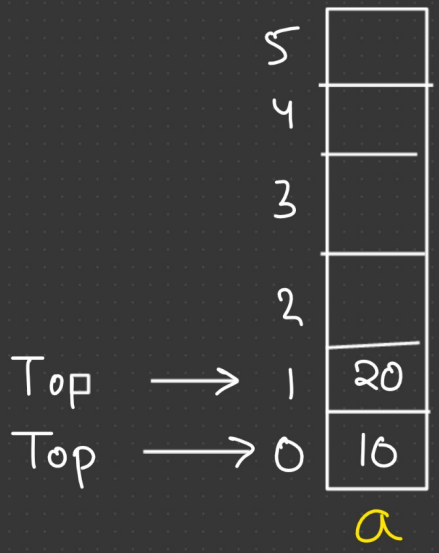
→ shifting & (size of array - 1)

Home work

Stack







Top = -1 → empty

push(int value)

pop()

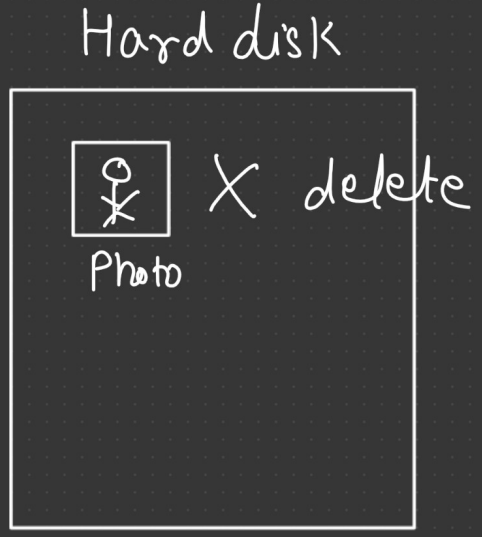
Restrict

```
Class  stack
{
  private:  int a[100];
            int top;

  public:
          stack ( )
          {
            top = -1;
          }

          void push (int x)
          {
              if (top == 99)
                  cout<<" Stack overflow";
              else {  top++;
                      a[top] = x;
```

```
    }    }
```

```
void pop()
{
   if( top == -1)
      Cout<<" Empty";

   else
        top--
  }
```

```
int peek()
{
   if( top == -1)
      Cout<<"empty";

   else return a[top];
}
```

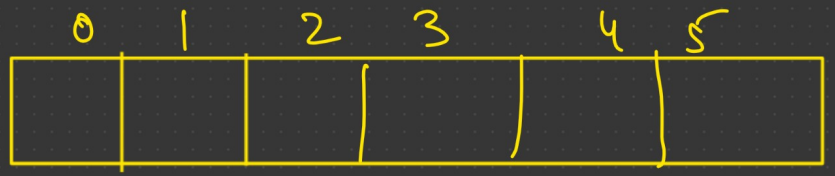


Data recovery

```
bool isEmpty()
{
    if ( top == -1)
        return true;
    else
        return false;
}
}
```

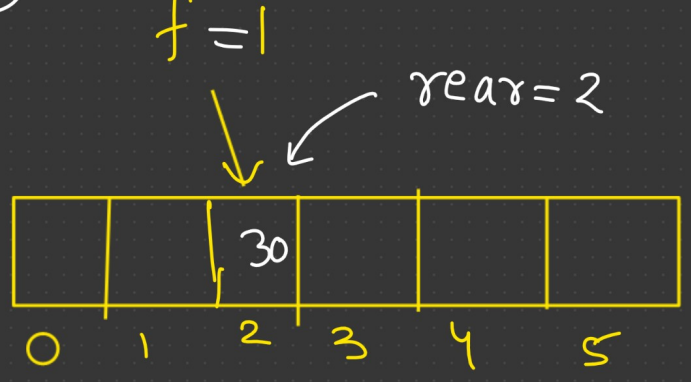
Queue

-



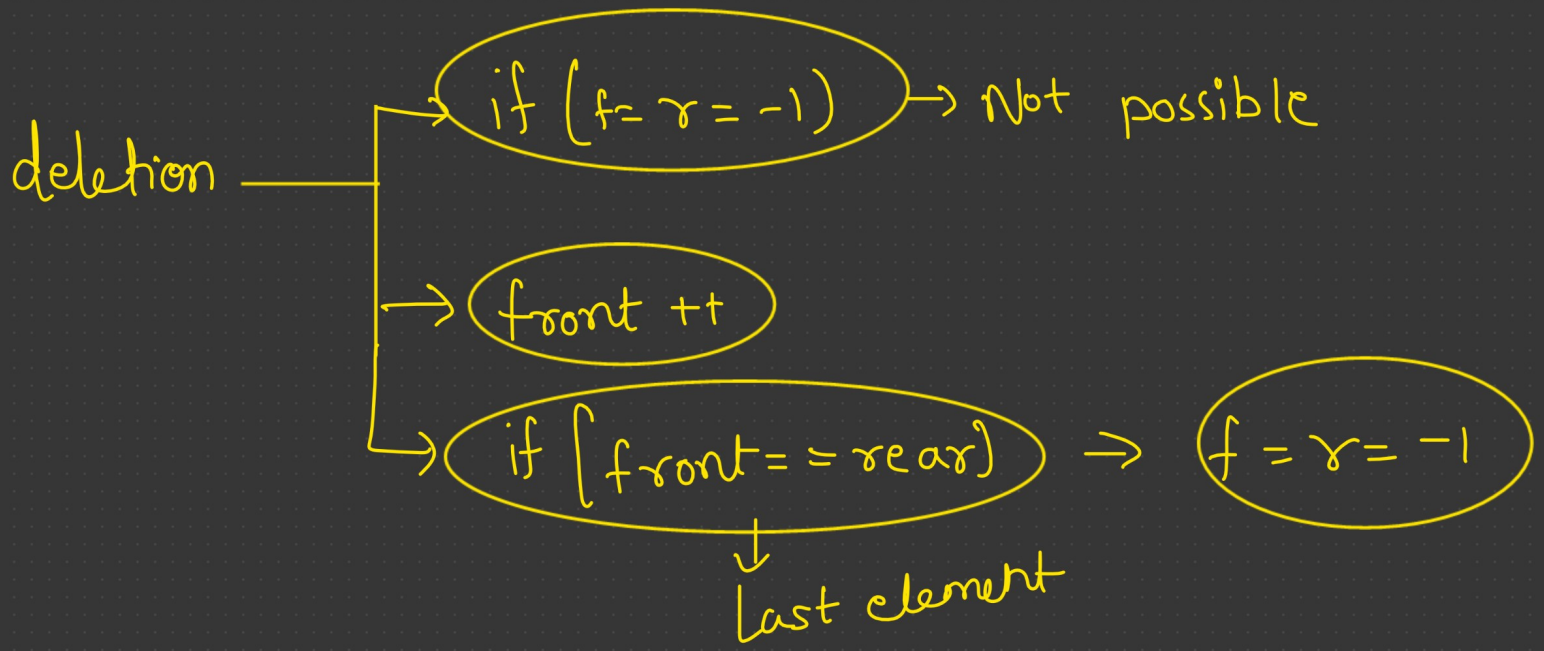
front
(delete)

rear
(insert)



empty → front == rear = -1

insertion →
- if front = rear = -1 → 0
- rear++
- Overflow → rear = n-1

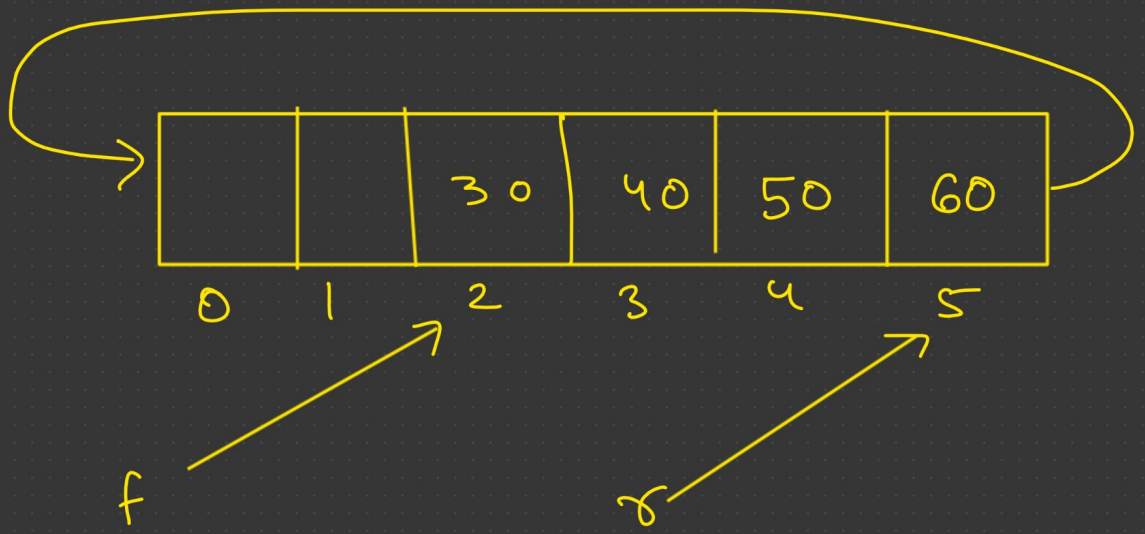

deletion
if (f= r = -1)
Not possible
front ++
if (front == rear)
f = r = -1
Last element

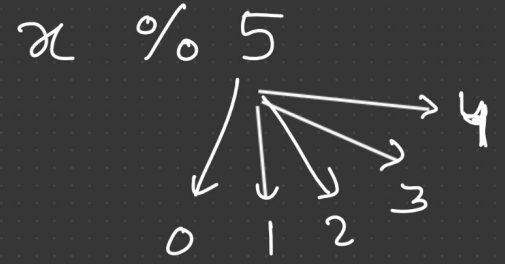
```
Class Queue
{
  private:
    int q[10], front, rear;

  public:
    Queue()
    {
      front = rear = -1;
    }

    void enqueue(int X)
    {
      if(rear == 9)
        cout<<"Overflow";
      else
      {
        if((front == -1) && (rear == -1))
```

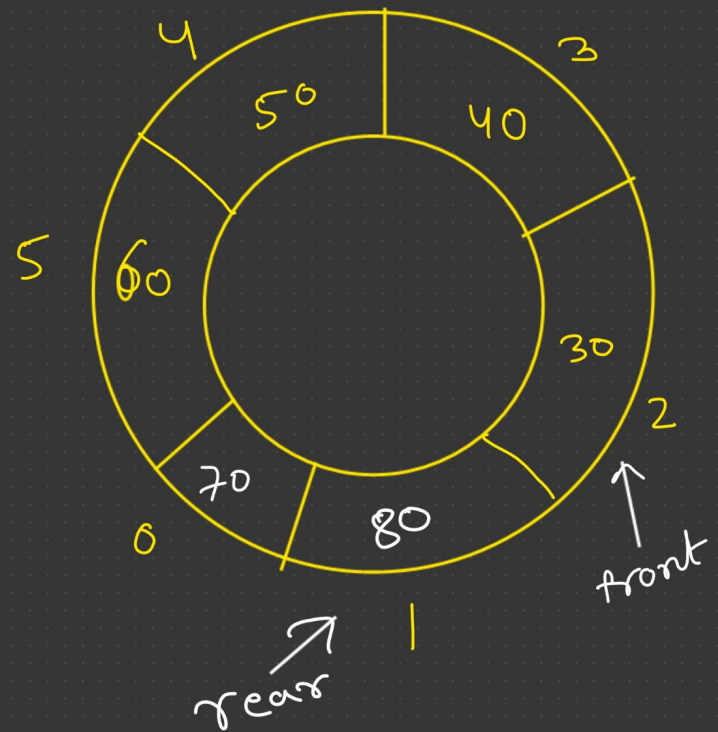
```
    {   front = 0;
        rear = 0
        q[front] = x;
    }
    else
    {
        rear++;
        q[rear] = x;
    }
}

void deque()
{
    if (rear == -1)
        cout<<"underflow";
    else
    {
        if (front == rear)
        {
            front = -1;
            rear = -1;
```

```
        else
            front++;
    }
};
```

Circular Queue



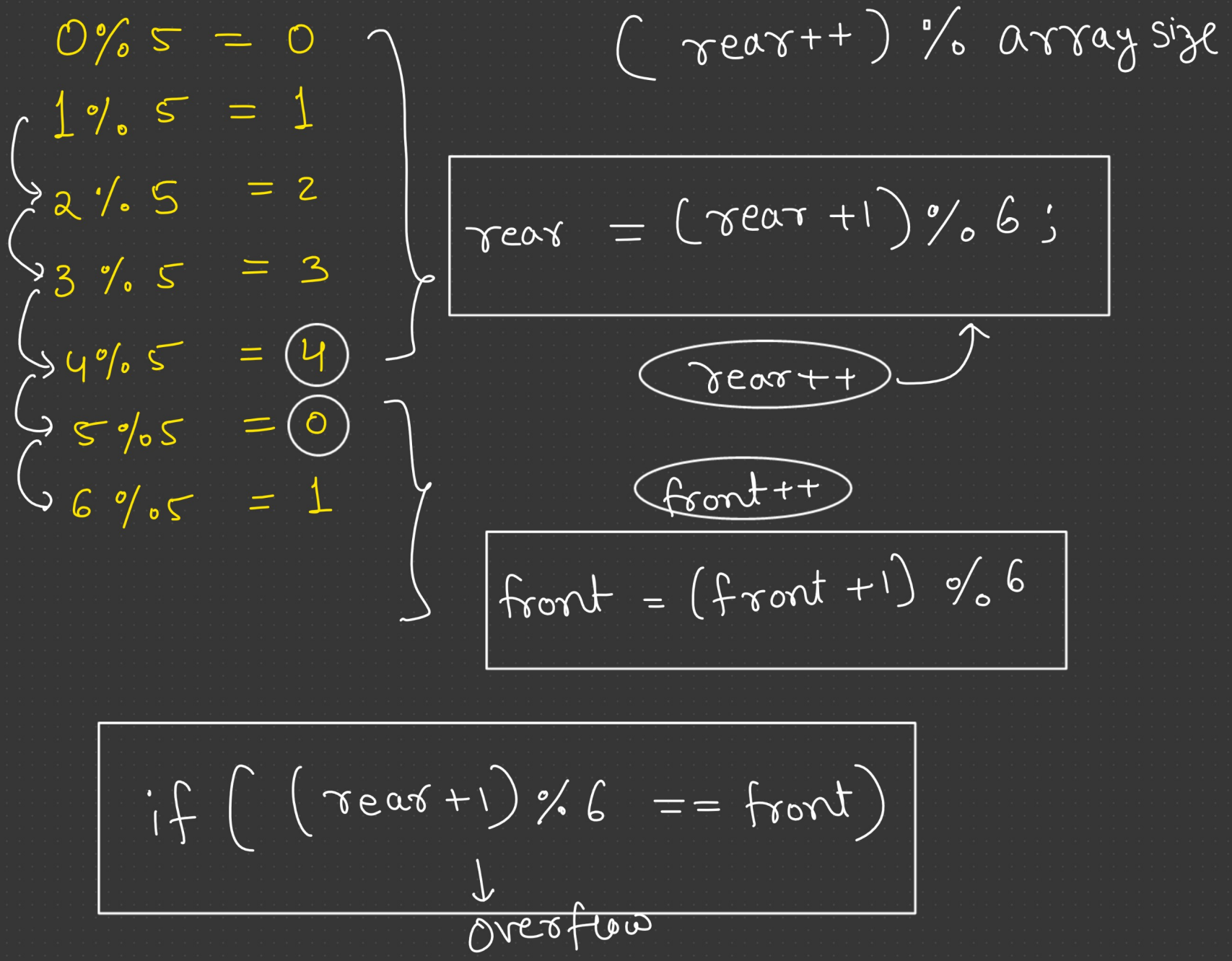




% 10

↓

[0 – 9]


0 % 5 = 0
1 % 5 = 1
2 % 5 = 2
3 % 5 = 3
4 % 5 = 4
5 % 5 = 0
6 % 5 = 1
(rear++) % array size
rear = (rear +1) % 6 ;
rear++
front++
front = (front +1) % 6
if ((rear +1) % 6 == front)
overflow

1) Minimum Pair code
2) Stack
3) Queue
4) Circular Queue
5) LC - 20 (valid Parant)
6) LC - 682 (Base Ball game)